

贷后信贷风控与处置

信衡 CreditSentry

从风险预警到授信处置
的多 Agent 可举证自主闭环

银行不缺预警系统。 缺的是预警之后那一段。

谁去取证、凭什么下结论、敢不敢自动执行、出了事怎么向监管交代——
这四个问题没答案，再准的预警也只能停在「给人看的建议」。

一位风险经理的一天

预警系统很勤快，人跟不上。

1,200+ 每日新增预警 重复推送与例行提醒占大头	4 核实一条要跨的系统 征信 · 司法 · 工商 · 流水	3 天 单案平均处置周期 取证、写报告、走审批	0 可复核的完整证据链 依据散在聊天记录与经验里
--	---	---	--

真风险淹在噪声里，误报又把正常客户抽贷断贷。

四个痛点，两种性质

前两条是效率问题，后两条是责任问题——责任问题解决不了，效率做得再好也上不了生产。

效率 · 01

预警过载

日均千级，逐条人工核实，客户经理被淹没。

效率 · 02

取证割裂

四套系统、四套口径、四套授权，证据分散且难固定。

责任 · 03

不敢自动执行

压降额度直接影响客户，错一次就是投诉与问责，于是宁可全人工。

责任 · 04

无法举证

内审问「当时凭什么这么判」，答不上来。

那为什么不直接上大模型？

因为信贷处置动作要动客户额度。不可复现，就不可审计。

让模型自由编排

同一个案子重跑十次，可能走出七条不同流程。

内审问「为什么这次多查了一步」——没人答得上来。

结论好坏先不论，过程本身就不可复核。

我们的做法

流程骨架用确定性状态机，12 条路由规则显式声明。

模型只在阶段内做判断，不决定流程走向。

每一次分支都带规则编号与版本，可直接读出来。

代价是它不够聪明——不会自己想出新流程。
但在信贷处置这个场景，这个交换很划算。

我们的答案

1 个 Manager + 1 个 Team Leader + 6 个职能 Worker 的端到端闭环



为什么是 6 个 Agent，不是 3 个或 10 个

把《商业银行内部控制指引》的「不相容职务分离」编译成权限矩阵，数量就数得出来。

5

+

1

=

6

权限等价类

目标函数对立拆分

职能 Worker

01

内部只读聚合

不触外部、不下结论

02

唯一 PII 取证触点

触点唯一，脱敏范围才可收敛

03 · 唯一拆分处

零工具纯推理

权限完全相同，因目标对立而拆

04

唯一写触点

全系统只有它能改额度

05

唯一审计视角

看得见全链路，动不了业务系统

合并任意两个会怎样： 定性者拿到取证工具 → 边查边下结论的确认偏差 · 定性与质疑合并 → 自己质疑自己，对抗坍缩 · 执行者兼审计 → 直接违反不相容职务分离

让 Agent 敢说「这不是风险」

比让它报警难得多。误杀正常客户会直接抽贷断贷、引发投诉与监管问责。

风险定性官

目标：论证**风险成立**。

零工具权限——刻意剥夺，防「边查边下结论」。

每条结论必须挂载证据编号，无源结论直接被拒。

对抗质疑官

目标：论证**风险不成立**。

输入完全相同、上下文完全独立、目标刻意对立。

对自动执行有一票**阻断权**。

质疑不可跳过

写在路由表里的硬约束，不交给模型判断。

两侧必须异构

绑定不同技术路线的模型。同源则「唱反调」变成自说自话，**配错直接拒绝启动**。

覆盖率参与裁决

未被质疑的主因不算通过——前者只是没人看过。

零人工，不等于无限自动化

风险等级 × 证据等级 × 敞口金额 × 可逆性 → 四档。纯规则，不联网、不问模型。

L0 只读诊断 全自主。打标、出建议，不碰业务系统。	L1 轻度干预 全自主。发通知、要材料，可撤回。	L2 影响授信 必须人工审批后执行。压降额度、追加担保。	L3 不可逆或大额 只出方案，系统永不执行。提前收贷、诉讼保全。
同一动作，不同档位 「压降 30%」在敞口 620 万时是 L2，敞口 2 亿时升为 L3。	失败方向 任何入参缺失或规则未命中，一律降为最高管控档，而不是「没匹配上就放行」。	审批被驳回 立即降级 L0 只读诊断，客户额度不被修改。	

865 组全组合遍历证明：不可逆动作在任何入参组合下都不会落进自主档。

三条链路，三种结局

A 与 B 是同一类主体、完全相同的三类信号，差别只在个案细节。

链路 A · 风险成立 涉诉 3 笔 · 集中转出 480 万 · 法代变更 三类信号相互印证，质疑逐条尝试反驳但均不成立。 L2 需审批 额度压降 30% + 追加担保 审批通过后执行，回执与审计留痕	链路 B · 风险不成立 同样三类信号 但逐笔下钻：涉诉已结案且我方为原告、转出对手方是稳定供应商、法代变更早于风险窗口。 L0 维持原状 出「加强监测」结论并留痕—— 「看过并决定不动」也必须有据可查	链路 C · 真实历史回溯 2017 年担保圈风险传染 时点冻结在 2017-04-01，只喂入该日之前的公开信息。 L3 只出方案 直接敞口 6.5 亿触发大额升档 系统全程不执行任何写操作
---	---	--

链路 B 才是技术难点。让 Agent 敢说「这不是风险」，比让它报警难得多。

链路 C · 真实历史案例回测

把时钟拨回 2017 年 4 月，它看出来了吗



系统当时的判定

风险成立 · 关注类

净未覆盖代偿敞口达直接敞口的 **2.02 倍**

但档位是 L3

直接敞口 6.5 亿触发大额升档，**只出方案不执行**。历史上该主体当时确有阶段性缓释（覆盖 54.6%）——**方向对不等于处置对**。

怎么保证没作弊

每条证据标注首次公开日并逐条校验；**历史结局在数据构造时即被摘出**，任何环节结构上取不到。

它现在就能打开，而且可以被当场检验

```
python3 poc/serve.py --open
```

零依赖、零云账号、零网络。

最有说服力的三个动作

- 点开任一证据 → 翻开原件

裁判文书、征信报告直接展开，并当场重算防篡改校验码

- 审批时它真的停下来等你

后端线程阻塞在那里，不是走过场的确认框。驳回 → 降级只读，额度未被修改

- 改一个案情要素 → 当场翻案

结论随之改变。改了却不变，说明它不是从证据推出来的

界面上看得到什么

左栏 预警池 · 判断方式 · 案情要素调节

中栏 五阶段流水线随处置推进逐段点亮；证据、对抗视图、闸门定档、执行回执按顺序长出来

右栏 办理过程 · 办理日志 · 取数留痕 · 岗位与权限

另外四个动作：并排看定性⇌质疑的逐条分歧 · 敞口改 2 亿看档位升级 · 让零权限岗位去改额度看它被拒且留痕 · 接自己的大模型 API

工作台与命令行是**同一套代码、同一份产出**，没有为演示另造一条链路。

一个系统的可信度，看它出问题时往哪个方向倒

三条失败路径，降级方向**恒为阻断**，不为放行。

失败 一

模型给不出合规输出

定性失败 → 判「证据不足」回退补证
质疑失败 → 判「质疑未完成」**直接阻断**

质疑器坏了不等于质疑通过。

失败 二

取证不足以定论

补证 ≤ 2 轮，仍不足则转人工——**且必须有交付物**。

出《取证任务清单》：为什么停、已查到什么、还缺什么、**每项找谁补**。

失败 三

人工审批被驳回

立即降级只读诊断，客户额度不被修改。

可在取数留痕里核对：**没有任何一次成功的写操作**。

我们把所有失败路径都调成了「停下来、说清楚、交给人」，
而不是「尽力而为地跑完」。

不是承诺，是可以现场跑的断言

每一条约束都由回归测试强制

74	906	865	13	36	0
安全边界回归 全绿	Skill 评估用例 全绿	闸门全组合遍历 穷举非抽样	接大模型的 故障模式回归	可逐条复核的 验收断言	第三方运行时 依赖

写权限唯一 只有一个角色能改额度	PII 触点唯一 只有一个角色能读征信与流水	越权尝试也留痕 被拒的调用同样进审计日志	配置零漂移 部署配置与权限矩阵逐字节比对
---------------------	---------------------------	-------------------------	-------------------------

其中带「含反向验证」的断言不只测「合法输入能通过」，还测「非法输入真的会被拒」——只测正向的约束等于没有约束。

逐条映射到赛题要求的能力

不是「用到了」，而是「用在哪、怎么验」。

能力	我们怎么用	怎么验证它真的成立
AgentTeams	1 Manager + 1 Team Leader + 6 Worker；身份、权限、模型绑定全部声明式	配置由权限矩阵生成，磁盘文件与矩阵逐字节比对，漂移即测试失败
Skill	16 个。领域判断 12 个全自研，通用云操作复用官方并标注开源等价替代	三件套（文档 + Schema + 评估集）全部由代码元数据生成；评估集是发布门禁
MCP	4 个 Server、17 个工具，覆盖信贷核心 / 征信 / 司法工商 / 账务流水	Mock 与真实接入零 Schema 差异；中央授权 + 审计留痕，越权调用被拒并留痕
RAG	监管政策库 + 历史案例库；六维查询改写、时点过滤、结果融合、经验回流	条款生效日 ≤ 案件时点，防知识维前视污染；每条命中记录由哪几维召回
可观测	10 类轨迹，对齐 OTel GenAI 语义；另加路由决策与对抗裁决两类专有记录	「为什么走了这条分支」直接读出来——带路由键、规则编号与版本
治理与网关	Nacos 版本化与灰度回滚；Higress 凭证透传与 PII 出站脱敏	凭证一律是引用而非明文，配置可进版本库、密钥不进配置层已预留，网关侧未落地

把「信贷」两个字去掉，还成立的部分

这是我们挑选开源内容的唯一标准。成立的才有被别人拿走的价值。

组件	说明	协议
Agent 拆分法则	数量由权限等价类推导的方法论	CC BY 4.0
EvidenceLedger	证据账本，六个里最通用的一个	Apache-2.0
RiskGate	四维执行闸门内核	Apache-2.0
CreditSkill Spec	Skill 规范三件套 + 评估门禁	Apache-2.0
4 个 MCP 契约	金融取数接口，含契约测试	Apache-2.0
案例回放数据集	含误报陷阱样例与反事实配对	CC BY 4.0

可平移到哪些场景

保险核赔 报案归并 → 单证取证 → 定损 ⇔ 反欺诈质疑 → 分级赔付

供应链金融 贸易背景真实性核验，处置动作换成放款闸门

工业设备运维 告警降噪 → 根因定位 ⇔ 反驳 → 远程复位可自主、停机必须人批

内容与账号风控 限流可逆、封禁不可逆，天然适配四维闸门

可迁移的是骨架：信号归并 → 证据固定 → 对抗式判定 → 按可逆性分级处置 → 留痕审计。

不可迁移的是判定口径——它绑定具体监管条款，换行业必须重写。

还没做的，写在明处

把差额写清楚，比事后被追问更经得起看。

已完成

6 Worker 的身份与配置声明 · 五阶段状态机与 12 条路由 · 16 个 Skill · 4 个 MCP Server · 证据账本与原文快照 · 三条端到端链路（含真实历史回溯）· 接大模型的输出契约层与失败策略 · 转人工交接单 · 风险处置工作台

尚未做

真实行内系统接入（仍为 Mock，接口按零差异设计）
生产级并发与多租户隔离
9 个 Skill 的评估集仍为种子集
Nacos 与 Higress 网关侧未落地
接大模型已在故障注入端点跑通全部 13 个模式，**但尚未在真实商业端点上跑过**

一处已知的标定局限

敞口升档阈值目前对所有客户分层用**同一套绝对值**。
在大型集团客户上，会把几乎所有可逆动作推到最高档。
机制是对的，标定不对。
这是跑真实案例才暴露的问题，如实记在这里而不是悄悄调参掩盖。

复赛计划

接入真实 Nacos 做灰度回滚演练 · Higress 落地凭证透传与 PII 脱敏 · 评估集扩至 100+ 标注案例 · 故障注入演练扩展到系统侧 · 敞口阈值按客户分层参数化 · 扩展至「贷后 + 反欺诈 + 法务」三 Team 协同

主指标：配对翻转率

同一主体、只改一个决定性因子，结论必须随之翻转。这个指标**无法靠「一律报警」或「一律放行」刷分**——两种偷懒策略的翻转率都是 0。

这套系统的价值不在于它跑得多快， 而在于它出问题时往哪个方向倒。

银行敢不敢让 Agent 动客户的额度，取决于三件事：结论有没有据、越权能不能

挡、出错会不会自己停下来。

这三件事我们都做成了可以现场检验的断言。

打开工作台

```
python3 poc/serve.py --open
```

跑三条链路

```
python3 poc/run_demo.py --all
```

跑安全回归

```
python3 poc/test_safety.py
```